

AI-SmartHome

Project Documentation & Development Checkpoint

Current checkpoint: Tested FastAPI + SQLAlchemy + SQLite backend with a virtual device simulator, repository-based persistence, in-memory device state, and working ON/OFF REST APIs. The current development phase is the React dashboard.

1. Project Overview

Purpose

AI-SmartHome is a virtual-first smart-home IoT platform designed to be developed and tested on Windows first, then deployed to Raspberry Pi with minimal changes to the business logic.

Main goal

Provide a clean smart-home architecture where React communicates with a FastAPI backend, the backend manages device state through a DeviceManager, and persistence is handled through SQLAlchemy and SQLite.

Development philosophy

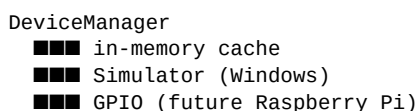
Keep hardware access behind an abstraction layer. The Windows simulator can later be replaced by Raspberry Pi GPIO without rewriting API routes or core business logic.

2. Target Architecture

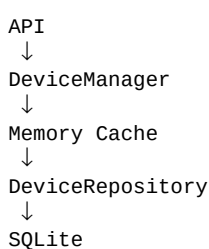
Application flow



Device hardware abstraction



Device state flow



Architecture rules

The simulator must not directly access SQLite. API routes must not directly access the database. DeviceManager should not contain raw SQL. Repository classes handle persistence.

3. Current Project Structure

Root

```
AI-SmartHome/  
  README.md  
  backend/  
  docker/  
  docs/  
  frontend/  
  venv/
```

Backend

```
backend/  
  .env  
  .gitignore  
  config.py  
  requirements.txt  
  app/  
    main.py  
    api/routes/  
      dashboard.py  
      devices.py  
      users.py  
      voice.py  
    core/  
    database/  
      base.py  
      connection.py  
      init_db.py  
      session.py  
    models/  
      command.py  
      device.py  
      log.py  
      user.py  
    repositories/  
      command_repository.py  
      device_repository.py  
      user_repository.py  
    schemas/  
      device.py  
    services/  
      ai/  
      device/  
        automation.py  
        gpio.py  
        manager.py  
        mqtt.py  
        scheduler.py  
        simulator.py  
        voice.py  
      speech/  
      tts/  
    utils/  
  tests/
```

Frontend planned structure

```
frontend/  
  package.json  
  vite.config.js  
  index.html  
  .env.development  
  src/  
    main.jsx
```

```

■■■ App.jsx
■■■ index.css
■■■ components/
■   ■■■ Header.jsx
■   ■■■ Sidebar.jsx
■   ■■■ StatusBadge.jsx
■   ■■■ DeviceCard.jsx
■   ■■■ RoomSection.jsx
■   ■■■ DeviceGrid.jsx
■■■ services/
■   ■■■ api.js
■■■ utils/
    ■■■ deviceUtils.js

```

4. Python Environment

Python version

Python 3.12.10 is the intended project version.

Virtual environment

E:\capstone\AI-SmartHome\venv

Backend directory

E:\capstone\AI-SmartHome\backend

Activation command

..\venv\Scripts\Activate

Backend startup

python -m uvicorn app.main:app --reload

Important working-directory rule

Run the backend from the backend directory. Running from the project root previously caused `ModuleNotFoundError` because the app package is under backend.

5. Backend Stabilization Completed

Files modified

backend/.gitignore; backend/requirements.txt; backend/app/database/session.py; backend/app/database/init_db.py; bac

Files created

backend/app/schemas/device.py; backend/app/repositories/device_repository.py; backend/tests/test_devices.py; root F

Major fixes

Corrected package imports; completed schema and repository modules; synchronized simulator/in-memory state with SQLite; registered the device router; replaced deprecated startup event handling with FastAPI lifespan; ignored SQLite database files in Git; added pytest and httpx test dependencies.

Verification

Backend tests were reported as 5 passed, 0 failed, 0 errors.

6. Database & Device Layer

Database

SQLite is used initially. Database initialization is idempotent and seeds seven default devices.

Device model

The device model concept includes: id, name, room, category, and status.

Repository

DeviceRepository encapsulates SQLAlchemy 2.x persistence operations.

Manager

DeviceManager orchestrates the in-memory cache, mock hardware/simulator, and repository writes.

Simulator

The simulator represents hardware behavior on Windows. It is intended to be replaceable by Raspberry Pi GPIO later.

Identifier rule

The API uses the exact device name, such as bedroom_light. Human-readable labels such as Bedroom Light can be used in the frontend.

7. Working REST API

Endpoints

```
GET /devices/  
POST /devices/{device}/on  
POST /devices/{device}/off
```

Swagger UI

<http://127.0.0.1:8000/docs>

Device list

<http://127.0.0.1:8000/devices/>

Manual verification

GET /devices was tested; bedroom_light was switched OFF using POST; GET confirmed OFF; bedroom_light was switched ON using POST; GET confirmed ON. The user confirmed the API was working properly.

Example response

```
[  
  {  
    "id": 1,  
    "name": "bedroom_light",  
    "room": "bedroom",  
    "category": "light",  
    "status": "ON"  
  },  
  {  
    "id": 2,  
    "name": "living_room_light",  
    "room": "living_room",  
    "category": "light",  
    "status": "OFF"  
  },  
  {  
    "id": 3,  
    "name": "kitchen_light",  
    "room": "kitchen",  
    "category": "light",  
    "status": "OFF"  
  },  
  {  
    "id": 4,
```

```
    "name": "fan",  
    "room": "living_room",  
    "category": "fan",  
    "status": "OFF"  
  }  
]
```

8. Frontend Phase

Technology

React + Vite + JavaScript/JSX + CSS + Lucide React + Fetch API.

API configuration

The planned development environment variable is `VITE_API_URL=http://127.0.0.1:8000`.

Backend integration

Only CORS middleware should be added/modified on the backend for normal frontend integration, unless an actual integration issue requires something else.

Dashboard requirements

Modern, professional, responsive dashboard with header, connection status, total/active devices and room count; sidebar navigation; dynamic device cards; room grouping; status badges; toggles; loading and error states; backend-offline handling; accessibility; and responsive desktop/tablet/mobile behavior.

Data rule

The backend is the source of truth. Devices and states must not be hardcoded in the frontend. The UI should update after a successful API response.

Planned API service

`getDevices()`, `turnDeviceOn(deviceName)`, `turnDeviceOff(deviceName)`.

Planned components

Header, Sidebar, StatusBadge, DeviceCard, RoomSection, DeviceGrid, plus App and global CSS.

Verification requirements

`npm run build` must succeed. Manual tests should cover GET /devices, ON/OFF actions, refresh persistence, backend unavailable state, and responsive layout.

Documentation

Root README and docs/Frontend_STATUS.md should be updated after implementation.

9. Technology Roadmap

Current

FastAPI + SQLAlchemy + SQLite + virtual device simulator + repository layer + in-memory device state + working REST API.

Next

React dashboard integration and frontend verification.

Later

AI/NLP, Gemini integration, speech-to-text, text-to-speech, MQTT, WebSockets, automation/scheduler, JWT authentication, Docker, and Raspberry Pi GPIO.

Cloud/DevOps direction

The architecture can later be containerized and extended toward deployment and IoT infrastructure without changing the core device-management abstraction.

10. Important Commands

Activate environment

```
cd E:\capstone\AI-SmartHome\backend
..\venv\Scripts\Activate
```

Run backend

```
python -m uvicorn app.main:app --reload
```

Run tests

```
..\venv\Scripts\python -m pytest tests
```

Frontend

After the React/Vite project is implemented: run npm install, then npm run dev; use npm run build for production-build verification.

11. Current Checkpoint

Status

Backend work is complete and verified according to the reported audit results.

What is working

FastAPI application startup, database initialization, device seeding, repository persistence, simulator/in-memory state handling, device REST endpoints, Swagger testing, and automated backend tests.

Current development target

Build and verify the React smart-home dashboard on top of the existing backend.

Do not redo

Do not rebuild the backend architecture or duplicate the database/device layer unless a concrete integration test reveals a defect.

Handoff statement

We have a tested FastAPI + SQLAlchemy + SQLite smart-home backend with a virtual device simulator, an in-memory device state layer, repository-based persistence, and working ON/OFF REST APIs; we are now building the React dashboard on top of it.

End of Project Documentation